

M365

TypeScript / React Coding Guidelines

Policy

Provided by ICIG Business Services

Document Owner	Möller, Philip	06/02/2026
Subject releaser	Weinreich, Tobias, Marangoni, Natascia, Wehe, Kevin, Schneider, Dennis	17/03/2026
System releaser	Nieendick, Michael	18/03/2026

Table of Contents:

1	Introduction	3
1.1	Purpose of Document	3
1.2	Scope	3
1.3	Related Documents.....	3
1.4	Terms and Acronyms	4
2	Rules for Working on Typescript Applications	6
2.1	Developer Commitments.....	6
2.2	Syntax.....	7
2.3	Coding Standards.....	14
3	Documentation	22
4	Document Changes	22
5	Appendix	23
5.1	Appendix 1: Overview of identifier styles	23
5.2	Appendix 2: Example of function description.....	24
5.3	Appendix 3: Example of a project structure	25

1 Introduction

1.1 Purpose of Document

As a developer of new and existing Typescript applications, both functional and safety aspects must be observed, as well as the readability of the code must be ensured.

This document supports you with:

- Rules for secure software development
- Rules to ensure the desired function

1.2 Scope

This document applies to all developers working on Typescript applications created or edited for ICI Group. This includes ICIG employees as well as external service providers.

1.3 Related Documents

Document title	Comments
SEC_P_1.01 Policy Information Security	Basic guideline for information security
SEC_P_1.02 Policy Data Protection	Basic guideline for data protection
SEC_P_1.03 - Co-applicable regulations for all directives	This document contains additional regulations on topics such as: Procedure in case of uncertainty/missing regulations or consequences in case of disregarding the regulations of this document.
SEC_P_2.01 - Safe handling of information	Relevant regulations for the secure handling of information for all employees
IT_Collaboration_PRC_001- M365 Development Process	Development process for M365 applications
SEC_P_2.17 - Safe development	The purpose of this document is to define basic requirements for the secure development of software and systems.
IT_QD_001 - Terms and Acronyms	This document provides the definition of Terms and Acronyms that are commonly used throughout ICIG Business Services.

1.4 Terms and Acronyms

The definition of terms and acronyms that are used in this document may be found in the central managed Terms and Acronyms document of ICIG Business Services

SharePoint Framework (SPFx)

"The SharePoint Framework (SPFx) is a page and web part model that provides full support for client-side SharePoint development, easy integration with SharePoint data, and extending Microsoft Teams and Microsoft Viva. With the SharePoint Framework, you can use modern web technologies and tools in your preferred development environment to build productive experiences and apps that are responsive and mobile-ready." [Microsoft¹]

Model

A model is a class/object which is used to represent a physical or logical object and its attributes.

Class Scope

Every object, variables and property inside of a class is in the class scope.

Local Scope/File Scope

Every object, variables, functions etc. located on the top level of a file, without the keyword `export` is in the local scope (file scope) of the code.

Global Scope

Every object, variables, functions etc. with the keyword `export` is in the global scope of the code.

Guard Claus:

A Guard Claus is an If statement that checks a termination condition. If the condition is fulfilled the current executed loop or function call will be skipped or terminated. To skip the current iteration of a loop you can use the keyword `continue`. To terminate the current iteration, you can use the keyword `break`. To terminate or quit a function call you can use the `return` statement.

UpperCamelCase:

UpperCamelCase is a naming style in which the first letter of each word is capitalized. Multiple words are concatenate to one word.

lowerCamelCase:

lowerCamelCase is a naming style in which the first letter of the word is not capitalized. The first letter of the following words is capitalized. Multiple words are concatenate to one word.

¹ <https://learn.microsoft.com/en-us/sharepoint/dev/spfx/sharepoint-framework-overview>

CONSTANT CASE:

CONSTANT_CASE is a naming style in which all letters are capitalized, and multiple words are split with an underscore.

M365
TypeScript / React Coding Guidelines
Policy

2 Rules for Working on Typescript Applications

This guideline contains rules for developing Typescript applications to meet safety standards and improve readability. The rules in this document are mandatory for all developers. Additional to this document the ruleset of the [TS dev website](https://ts.dev/style/)² **should** be applied.

2.1 Developer Commitments

#	Rule	Tips
2.1.1	All source code must be checked into the ICIG BS source code management system in compliance with the development process. This also includes administrative scripts that are not executed regularly.	
2.1.2	During the code review, the reviewer must check against the rules in this document.	
2.1.3	It must always be ensured that the user cannot adjust any configurations to security-relevant settings.	
2.1.4	TypeScript must be only used for the development of SPFx applications.	

² <https://ts.dev/style/>

M365
TypeScript / React Coding Guidelines
Policy

2.2 Syntax

2.2.1 Identifiers

#	Rule	Tips
2.2.1.1	Identifiers must use only ASCII letters, digits, underscores (for constants and structured test method names), and the '\$' sign.	Each valid identifier name is matched by the regular expression <code>[\$\w]+</code>
2.2.1.2	Classes must be named in UpperCamelCase-Style	<code>class UserData</code>
2.2.1.3	Interfaces must be named in UpperCamelCase-Style	<code>interface IBasicUser</code>
2.2.1.4	Types must be named in UpperCamelCase-Style	<code>type UserData</code>
2.2.1.5	Enums must be named in UpperCamelCase-Style	<code>enum WorkflowState</code>
2.2.1.6	Variables must be named in lowerCamelCase	<code>const userAge</code> <code>let inputText</code>
2.2.1.7	Properties must be named in UpperCamelCase	<code>public UserMail: string</code>
2.2.1.8	Parameters must be named in UpperCamelCase, if it is a property and in lowerCamelCase, if it is a variable	<code>constructor (UserMail: string)</code> <code>public sendMail(mailAddress: string)</code>
2.2.1.9	Functions must be named in lowerCamelCase	<code>public sendMail(mailAddress: string)</code>
2.2.1.10	Methods must be named in lowerCamelCase	<code>public sendMail(mailAddress: string)</code>
2.2.1.11	Global constant values must be named in CONSTANT_CASE	<code>const FROM_MAIL</code>
2.2.1.12	Enum values must be named in CONSTANT_CASE	<code>enum State {</code> <code>NOT_STARTED = "NOT_STARTED",</code> <code>STARTED = "STARTED"</code> <code>}:;</code>
2.2.1.13	Files of TypeScript must be named in UpperCamelCase.	<code>DocumentProperty.ts</code>

M365
TypeScript / React Coding Guidelines
Policy

2.2.1.14	Folder with TypeScript code must be named in lowerCamelCase.	For example, stateModels
2.2.1.15	An Identifier should not have more than 40 letters.	Example for the maximum length of an identifier: <code>let numberOfVeryImportantPeopleOfThisCompany</code>
2.2.1.16	Abbreviations like acronyms in names should be treated as whole words.	Use <code>loadHttpUrl</code> not <code>loadHTTPURL</code> .
2.2.1.17	Identifiers must not use “_” as a prefix or suffix except from private methods (see [2.2.2.6]). This also means that _ must not be used as an identifier by itself (e.g. to indicate a parameter is unused).	

2.2.2 Naming of Methods/Functions

In the following table the word “**function**” is used for functions as well as methods.

#	Rule	Tips
2.2.2.1	Functions must be named accordingly to their action	
2.2.2.2	The name of a function should be structured as following: “[action][object name]” or “[action][object name]by[filter/selection attribute]”	<pre>public sendMail(mailAddress: string) public sendMailByGroupMembers(groupName: string)</pre>
2.2.2.3	Each action represents a special key word that should appear in the method name (get, create, set, update, delete or is)	Function for loading a user: <code>getUser()</code> Function for creating a user: <code>createUser()</code> Function for updating a user: <code>updateUser()</code> Function for deleting a user: <code>deleteUser()</code>
2.2.2.4	If a function returns a Boolean value, its name must use the action key word “is”.	Function for checking if a user is active: <code>isUserActive()</code>

M365
TypeScript / React Coding Guidelines
Policy

2.2.2.5	If the filtering/selection action of a function can be performed in multiple ways, the filter or selection attribute should be part of the function name.	Function for deleting a user: <code>deleteUserById()</code> Function to get an Item by Name <code>getItemByName()</code>
2.2.2.6	Private methods must be named with a leading underscore.	<code>private _formateValue(value: string)</code>

2.2.3 Naming of Variables, Properties and Parameters

In the following table the word “**variable**” is used for all kinds of variables (like properties and parameters).

#	Rule	Tips
2.2.3.1	Variables must be named according to the object or information that they contain.	A variable that contains the age of an employe should be named <code>age</code> or <code>employeeAge</code>
2.2.3.2	The name of a variable should not contain abbreviations like “obj” or “num” to indicate the type of variable.	Do not use <code>numOfUser</code> . Use instead <code>numberOfUser</code>
2.2.3.3	Variables that are used to count instance of something or contains the number of something, must be named with starting “numberOf”.	Variable to count user: <code>numberOfUser</code>

2.2.4 Structure of Function-/Method-Descriptions

In the following table the word “**function**” is used for functions as well as methods.

#	Rule	Tips
2.2.4.1	Every function should have a description to identify all important information of the function	You can find an example for a function description in appendix 4.1
2.2.4.2	The following parts should be included in a function description: - DESCRIPTION - PARAMETERS - OUTPUT - NOTE	<pre>/** * DESCRIPTION A brief description of the functionality and particularities of the function * PARAMETERS Lists all parameter of the function (see rule 2.2.4.3) * OUTPUT The type of return value. It also can contain more information about the return value is self. * NOTE General information like version, author and modified date, as well as a change log with a short description of the last changes (see rule 2.2.4.4) **/</pre>
2.2.4.3	Every parameter of a function should be listed in the function description with the following structure: @param [parameter name] [parameter description]	<pre>* PARAMETERS @param item the SharePoint list item</pre>

2.2.4.4	The section NOTE should contain the following information: - Version - Author - Modified Date - Change Log	<pre>* NOTE Version: 1.0 Author: Alan Turing Modified Date: 1945-05-08 Change Log: Added decryption function</pre>
2.2.4.5	If a parameter of a function has a default value, the parameter description must contain the effect of this default value.	

M365
TypeScript / React Coding Guidelines
Policy

2.2.5 Styling of Code

#	Rule	Tips
2.2.5.1	One tabulator (tab) must be four spaces long.	<code>leftSideTab rightSideTab</code>
2.2.5.2	After the identifier it should not be a space between the colon, and it should be a space after the colon before the type.	<code>public Name: string</code>
2.2.5.3	The opening curly bracket should be in the same line as the code a head and separated with one space. The closing curly bracket should be in a new line under the last line of code.	<pre>public class User { [...] }</pre>
2.2.5.4	Statements like <code>catch</code> , <code>finally</code> and <code>else</code> should be placed in the same line after the closing curly bracket.	<pre>try { [...] } catch (error) { [...] }</pre>
2.2.5.5	Code between curly brackets must be indented with one tab.	<pre>public class User { public Name: string }</pre>

M365
TypeScript / React Coding Guidelines
Policy

2.2.5.6	To organize your code into visible sections, only one empty line should be place between the two blocks of code. Closing curly brackets should not be separated by an empty line.	<pre>public class User { public Name: string constructor (Name: string) { [...] } }</pre>
2.2.5.7	Comments must be placed in an own line and a space after the two slashes.	<pre>public class User { // This class is a lightweight user object. public Name: string }</pre>
2.2.5.8	Module names in a destructuring import statement must be sorted alphabetical.	<pre>import { Label, Link, Spinner } from 'office-ui-fabric-react'</pre>
2.2.5.9	Imports must be sorted due to the following order: 1. side effects imports 2. modules imports 3. default imports 4. destructuring imports	<pre>import "@pnp/sp/webs"; import * as React from 'react'; import store from "../redux/Store"; import { Label, Link, Spinner } from 'office-ui-fabric-react';</pre>

M365
TypeScript / React Coding Guidelines
Policy

2.3 Coding Standards

2.3.1 General Standards

#	Rule	Tips
2.3.1.1	TypeScript-files must be encoded in UTF-8 .	
2.3.1.2	The indicator var must not be used.	
2.3.1.3	Variables, Properties, Parameters, Functions and Methods must correspond to a distinct type whenever possible.	That helps the readability and the type safety.
2.3.1.4	The usage of the type any must be avoided whenever possible.	
2.3.1.5	The application should use the newest stable version of TypeScript.	
2.3.1.6	All static information like paths, usernames or group names that need to be configured by the user must be set due to the property pane of the application.	
2.3.1.7	All static information like paths, usernames or group names that do not need to be configured by the user, must be stored in a configuration file.	
2.3.1.8	To check if a string is empty, they must be checked against null , undefined and the lengths of the string equals zero.	<pre>if (test===undefined test===null test.length===0)</pre>
2.3.1.9	To determine the language of the user the application must use the SharePoint user localization.	

M365
TypeScript / React Coding Guidelines
Policy

2.3.1.10	The three expressions of a for statement must only be used to manage the runtime of the for loop. The expressions must not have side effects.	
2.3.1.11	The expression of a while statement must only be used to manage the runtime of the while loop. It must not have side effects.	

2.3.2 Project Structure

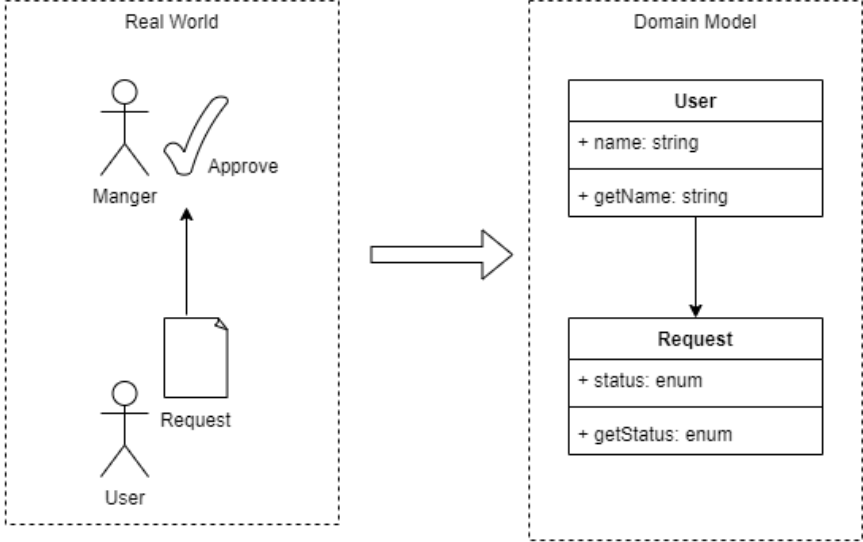
Appendix 4.3 shows an example of a project structure.

#	Rule	Tips
2.3.2.1	The TypeScript project must have a “.gitignore” file on the top level of the project.	
2.3.2.2	All corresponding files of the SPFx solution must be stored in a folder named “SPFX”.	
2.3.2.3	Code of other parts of the application/project must be stored in own folder on the top level of the project. Each topic has its own folder.	For example, there is one folder for a Power Automate workflow, and another folder for the setup script.
2.3.2.4	All corresponding source code of the SPFx solution must be stored in the “src” folder of the “SPFX”-folder.	
2.3.2.5	The “src” folder in the “SPFX” folder must contain the following folder: - constants - enums - interfaces - models	

M365
TypeScript / React Coding Guidelines
Policy

	- redux - services - stateModels - webparts	
2.3.2.6	If there are components of the SPFx solution, which are reusable in multiple webparts or at multiple points in one webpart, they should be stored in a folder named “components”, which is located inside of the “src” folder.	
2.3.2.7	All static values should be stored in a file “StaticValues.ts” which is in the folder “constants”.	
2.3.2.8	The information about list and column names must be stored in a file “SpListInfos.ts” which is in the folder “constants”.	

2.3.3 Code Structure

#	Rule	Tips
2.3.3.1	The application must be structured corresponding to the domain model. That means the code is structured into classes/objects and interfaces representing real or logical objects from the use case of the application.	
2.3.3.2	The code of the application must follow the rules of object orientated programming.	
2.3.3.3	Different components of the application should be separated in different files.	
2.3.3.4	Styles values for graphic objects should be stored in a ".css" file and referent due to the style class name.	

M365
TypeScript / React Coding Guidelines
Policy

2.3.4 Algorithmic Procedure

#	Rule	Tips
2.3.4.1	All algorithms should use the smallest amount of resources that's possible.	
2.3.4.2	To run a part of the code for a fixed number of times, a for loop must be used.	
2.3.4.3	To run a part of the code an unknown number of times, a while loop must be used.	
2.3.4.4	To run a part of the code an unknown number of times but at least once, a do-while loop must be used.	

2.3.5 Usage of Guard Clauses

#	Rule	Tips
2.3.5.1	To improve the readability and the understandability, guard clauses should be used instead of nested if statements.	
2.3.5.2	Nested if statements should be avoided whenever possible.	

M365
TypeScript / React Coding Guidelines
Policy

2.3.6 Types of Conditions

#	Rule	Tips
2.3.6.1	An if or if-else statement should be used if there are no more than three conditions to check.	
2.3.6.2	A switch-case statement should be used if there are more than three conditions to check.	
2.3.6.3	The Ternary operator should be used if the condition is simple (only one or two connected conditions) and the result of the condition is only a return value.	

2.3.7 Error handling

#	Rule	Tips
2.3.7.1	To protect the application from crashes, possible errors must be handled or checked with guard clauses.	
2.3.7.2	Any occurring error must be made visible for the end-user	
2.3.7.3	Any error that is displayed to the end-user must describe the error	Don't use general error texts like "An unexpected error has occurred", but describe the error "User not found", "Insufficient permissions" or "a network connection to the webservice could not be established"

2.3.8 Visibility and Scope

#	Rule	Tips
2.3.8.1	If class properties don't special getter or setter, the property should be public, and no getter or setter are be used.	
2.3.8.2	Code in global scope should be avoided.	

M365
TypeScript / React Coding Guidelines
Policy

2.3.8.3	The smallest scope possible should always be used.	
2.3.8.4	If code is in global scope, it must be used by another part of the application.	
2.3.8.5	Variables in global scope must not use <code>let</code> .	
2.3.8.6	Changeable variables which need to be global must be stored in the context of the web application.	
2.3.8.7	Accesses to the context must be protected against simultaneously or asynchronous accesses.	
2.3.8.8	To protect the context against simultaneously or asynchronous accesses, the application must be using the redux framework.	

2.3.9 Imports and Exports

#	Rule	Tips
2.3.9.1	Applications must only import modules that are actively used.	

2.3.10 Usage of React in SPFx

#	Rule	Tips
2.3.10.1	The framework React must be used for the development of SPFx applications.	
2.3.10.2	React classes must not be used in the application, accept from the webpart class.	
2.3.10.3	React functions components should be used in the application.	

M365
TypeScript / React Coding Guidelines
Policy

2.3.11 Usage of GLB.SPFXCore in SPFX

#	Rule	Tips
2.3.11.1	The GLB.SPFXCore should be used for SPFX applications.	There are many helper methods. (e.g.: checking null or placeholder replacement)
2.3.11.2	To check if a string is empty, the method <code>isStringUndefinedOrNullOrEmpty()</code> of the GLB.SPFXCore should be used.	

2.3.12 ESLint rule set from Microsoft

#	Rule	Tips
2.3.12.1	The code of the web application must follow the ESLint rule set from Microsoft.	
2.3.12.2	For SPFX applications the preinstalled ESLint rule set must be used.	
2.3.12.3	For non-SPFX applications ESLint must be installed, and it must be used the default rule set of Microsoft.	ESLint rule set of Microsoft (https://github.com/microsoft/eslint-plugin-sdl)

2.3.13 Usage of Fluent UI in SPFX

#	Rule	Tips
2.3.13.1	Fluent UI should be used for all graphic components of the application.	
2.3.13.2	For displaying error messages Fluent UI must be used with the standard functionality.	

3 Documentation

The documentation is archived on SharePoint.

4 Document Changes

The document history and the changes that have been performed to former versions of this document are listed in the following table:

Date	Version	Author	Reviewer	Document Changes
19.01.2026	3.0	Philip Möller	See subject releaser and system releaser listed at the beginning of the document	Changed rules 2.2.4.1 – 2.2.4.4 to “should” rules Adjusted rule 2.3.7.1 Added rule 2.3.7.2 and 2.3.7.3
03.02.2025	2.0	Philip Möller	See subject releaser and system releaser listed at the beginning of the document	Added SEC_P_2.17 as related document Corrected typos Added IT_QD_001 as related documents. Adjusted effective date
03.09.2024	1.0	Philip Möller	See subject releaser and system releaser listed at the beginning of the document	Initial document

5 Appendix

5.1 Appendix 1: Overview of identifier styles

Style	Category
UpperCamelCase	class / interface / type / enum / decorator / type / parameters
lowerCanelCase	variable / parameter / function / method / property / module alias
CONSTANT_CASE	global constant values, including enum values
#ident	private identifiers are never used.

5.2 Appendix 2: Example of function description

```
/**
 * DESCRIPTION
 *   Replaces all placeholders ({{key}}) to the correct value
 *
 * PARAMTERS
 *   @param item the item
 *   @param value the value
 *   @param targetDateFormat the format for dates (default DD.MM.YYYY)
 *   @param defaultValue the default value for fields that are undefined or null
 *
 * OUTPUT
 *   string
 *
 * NOTE
 *   Version: 1.0
 *   Author: Philip Möller
 *   Modified Date: 2023-05-23
 *   Change Log: Added function to replace a placeholders ({{key}}) to the corect value
 */
```

Figure 1 Description of the function placeholderReplacement() of the SpFxCore

5.3 Appendix 3: Example of a project structure



Figure 2 example folder structure of a SPFX project